



AHEAD Online

Data Structures and Algorithms

Geetha Lekshmy V

Harsha Raj

Department of Computer Science
Amrita Vishwa Vidyapeetham

Objectives

Circular Linked List

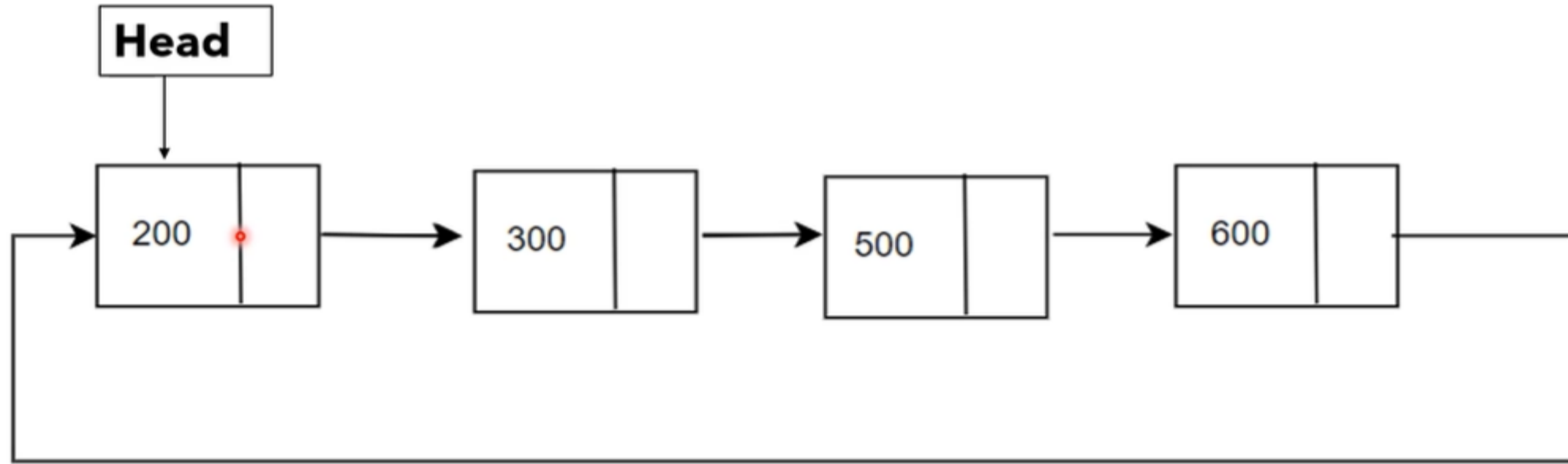
CLL - operations



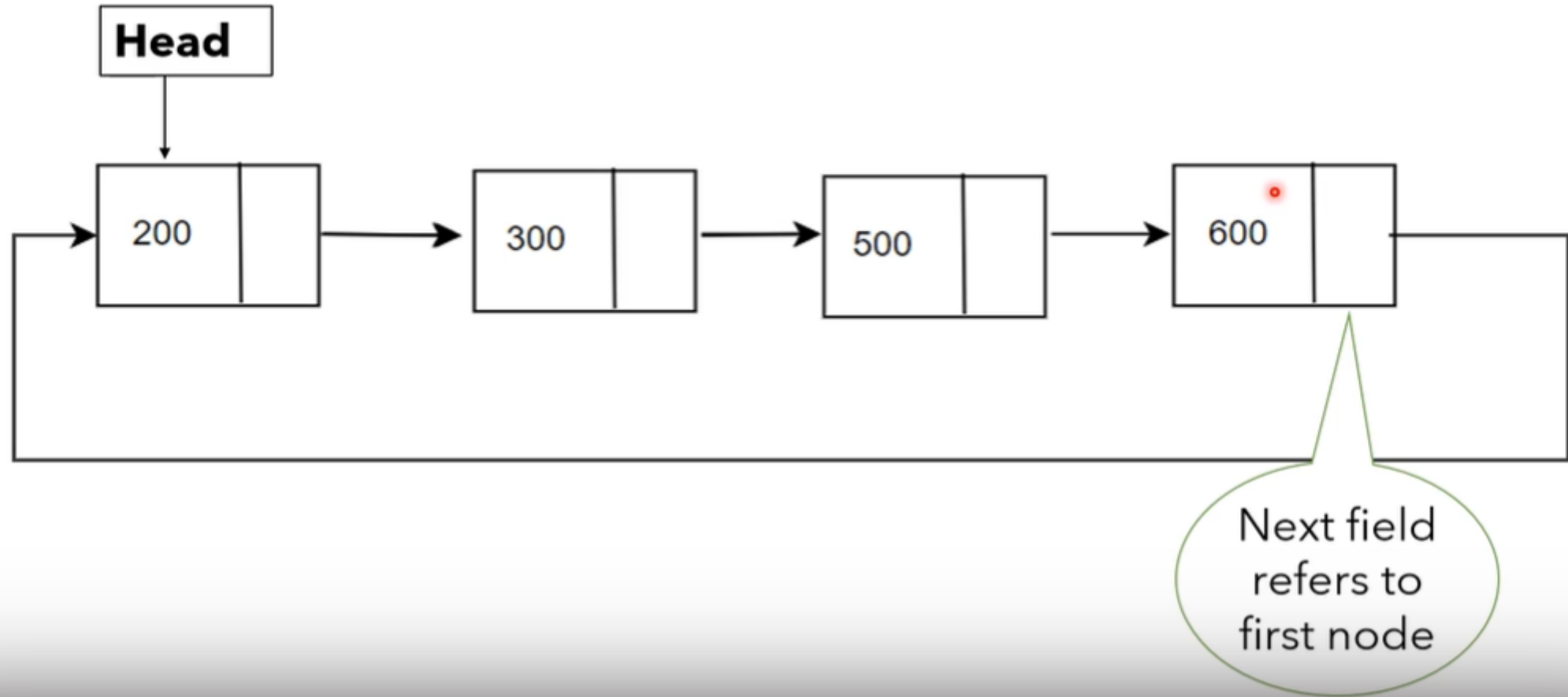
Circular Linked List

- Next field of last node refers to first node.
- No null value in next address field of any node.
- Head reference refers to first node.
- How list can be traversed??

Circular Linked List



Circular Linked List



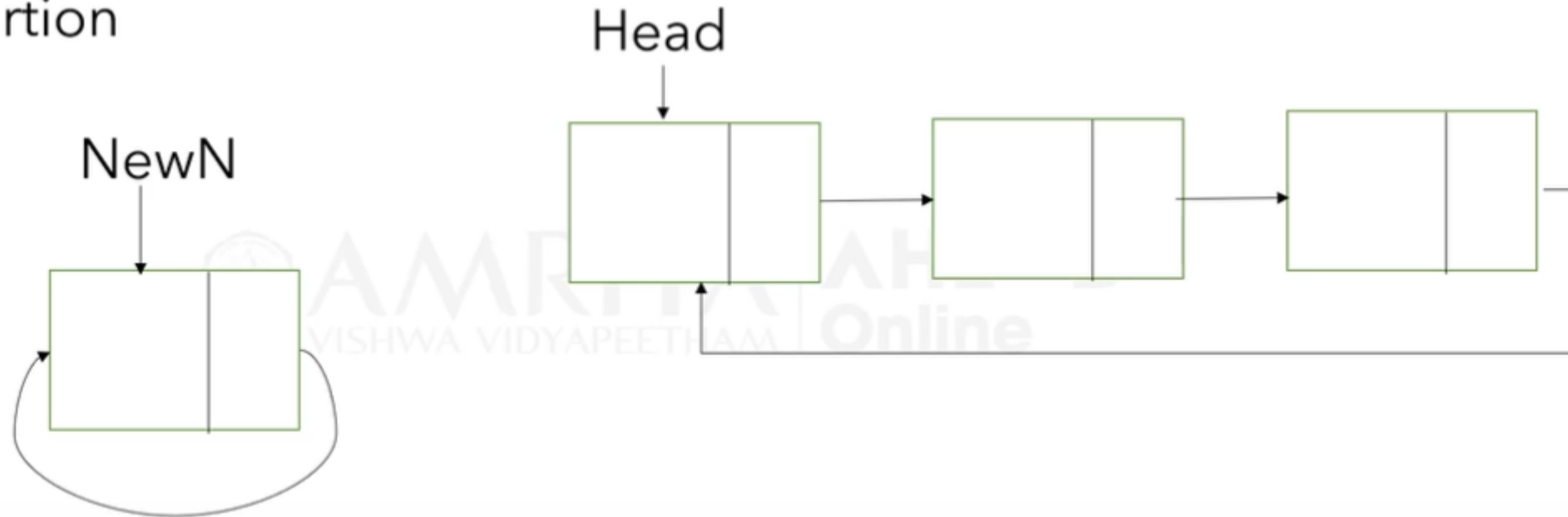
Circular Linked List

- Next field of last node refers to first node.
- No null value in next address field of any node.
- Head reference refers to first node.
- How list can be traversed??
 - In a loop, start from head, visit each node, print its contents and stop when you reach the node with next address field as head.
 - Print the contents of the last node.

Circular Linked List

Operations – Insertion, Deletion, Traversal

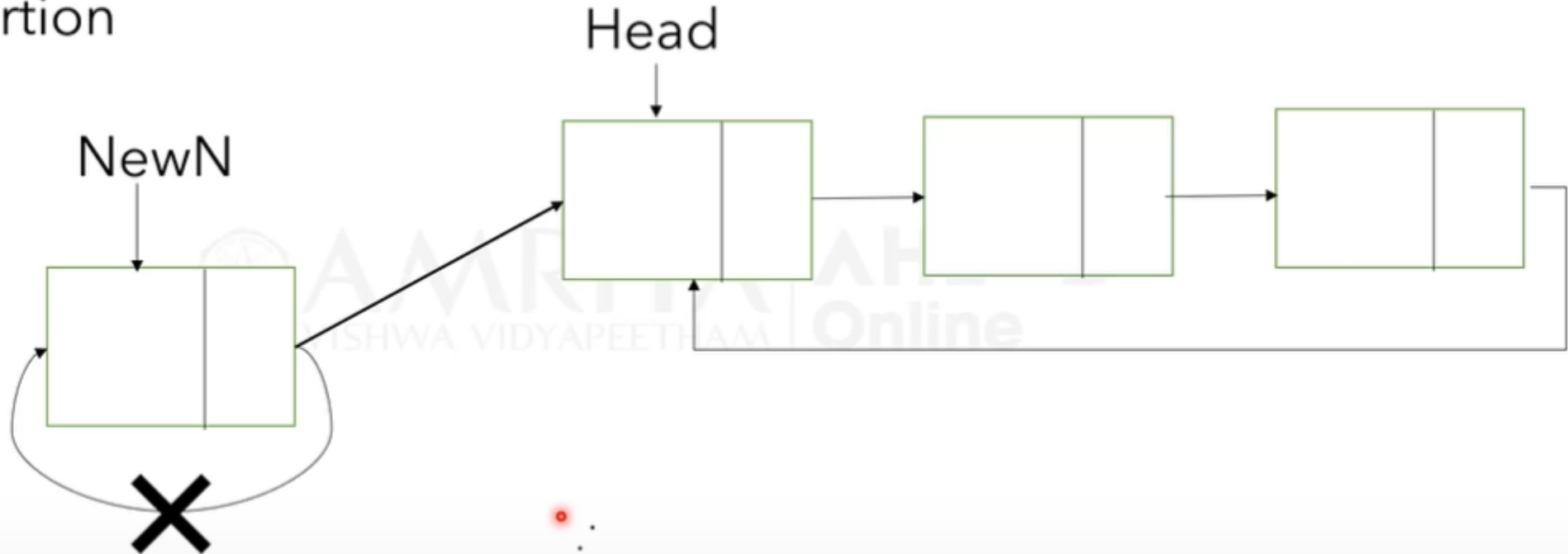
- Insertion



Circular Linked List

Operations – Insertion, Deletion, Traversal

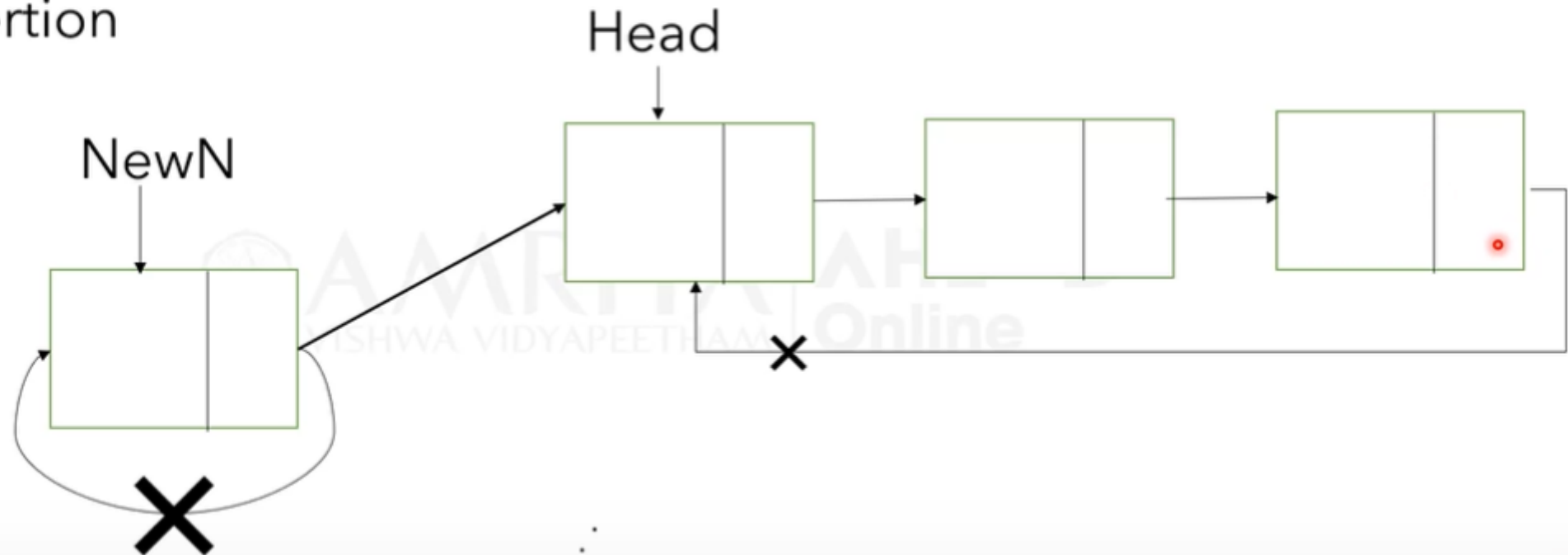
- Insertion



Circular Linked List

Operations – Insertion, Deletion, Traversal

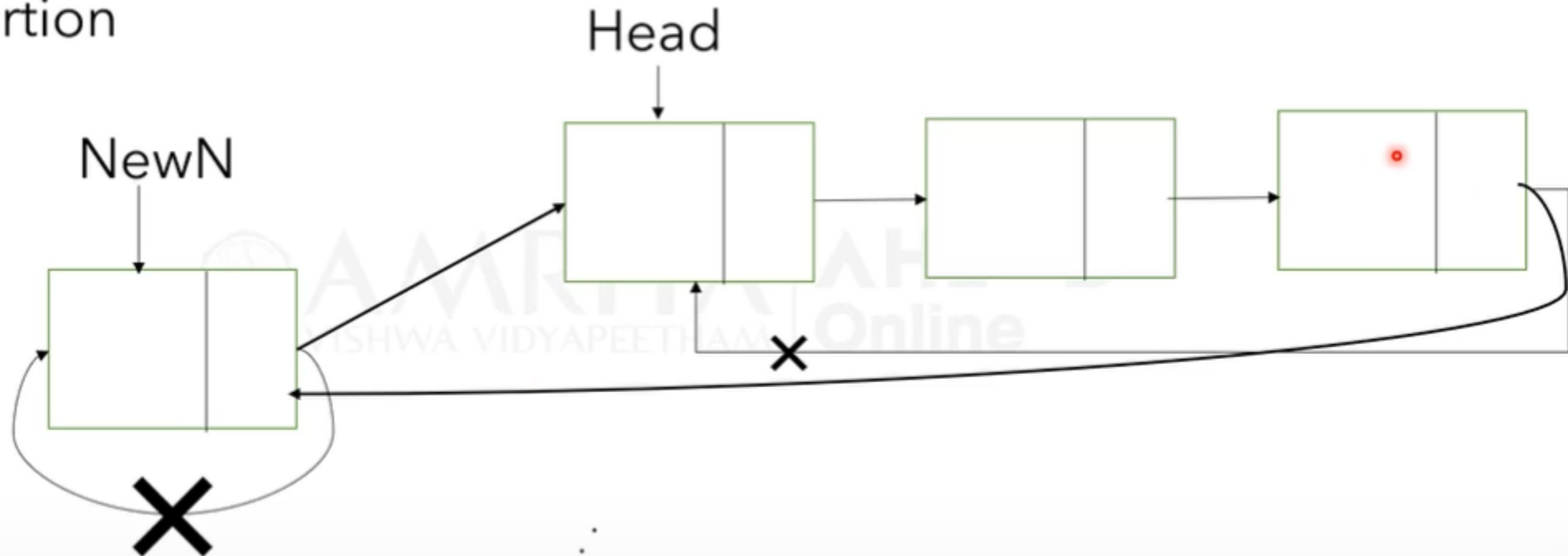
- Insertion



Circular Linked List

Operations – Insertion, Deletion, Traversal

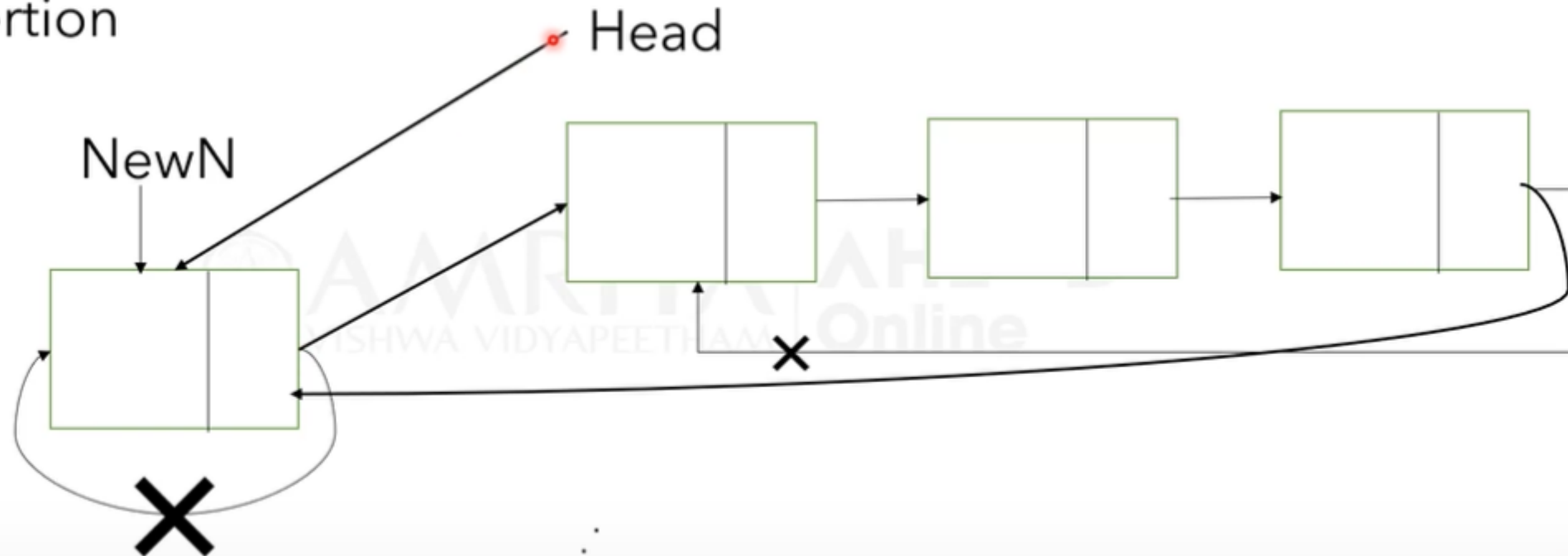
- Insertion



Circular Linked List

Operations – Insertion, Deletion, Traversal

- Insertion



Circular Linked List

- Insertion

```
public void insert_to_head(int d)
```

```
{  
/*if head is null then create a new node and set its next field referring to  
itself*/
```

```
    CNode cur;
```

```
    if(head==null) {
```

```
        head = new CNode();
```

```
        head.setData(d);
```

```
        head.setNext(head); //next field refers to itself
```

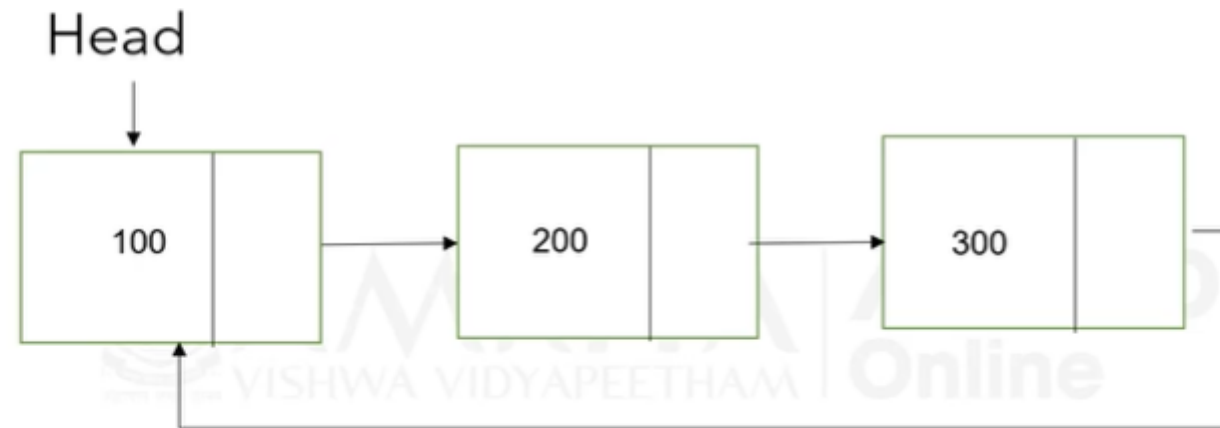
```
    }
```

Circular Linked List

```
else {  
    /*if head is not null, Create a new node and set its next field referring to head*/  
    CNode t=new CNode(); //CircularList Node  
    t.setData(d);  
    t.setNext(head); //set the next field to current value of head  
    cur=head;  
    while(cur.getNext()!=head) { //traverse and find out the last node  
        cur=cur.getNext();  
    }  
    cur.setNext(t); //set the next field of last node to newly created node  
    head=t; // reset head to new node  
}
```

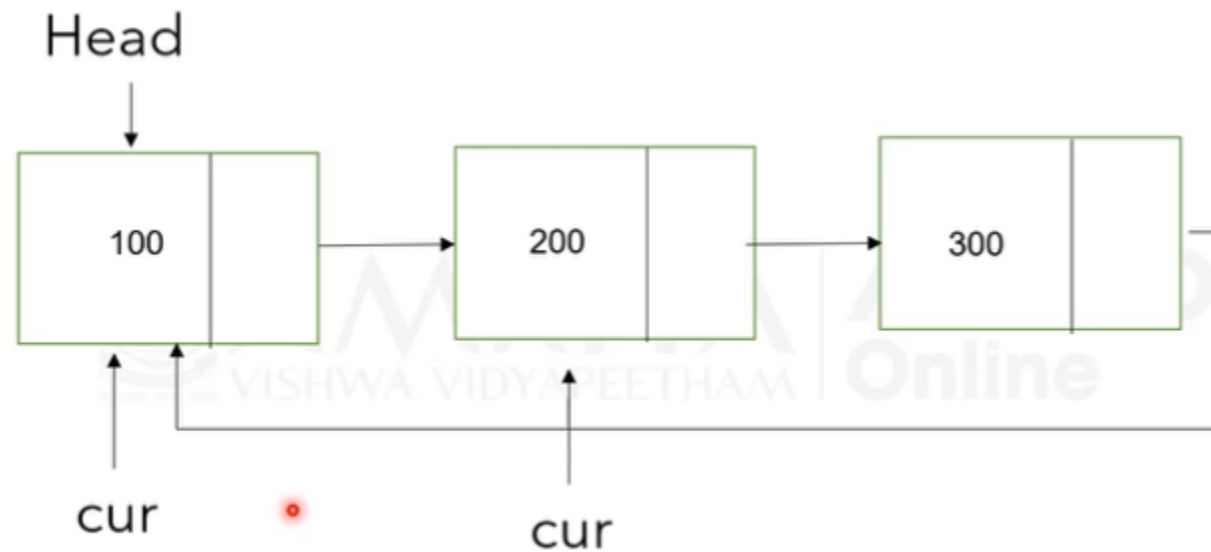
Circular Linked List

Operations – Traversal



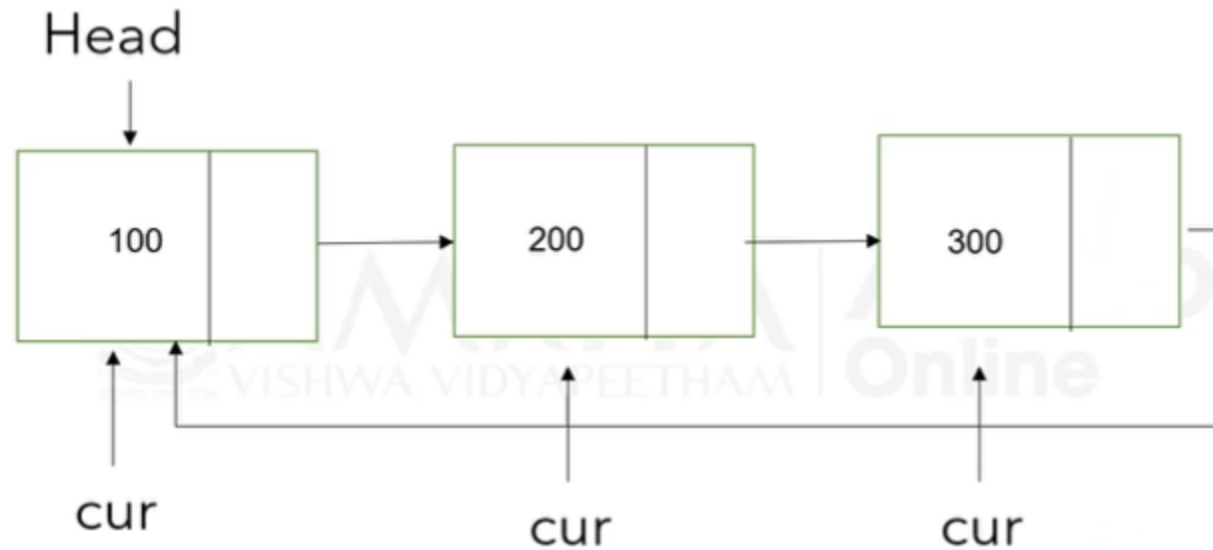
Circular Linked List

Operations – Traversal



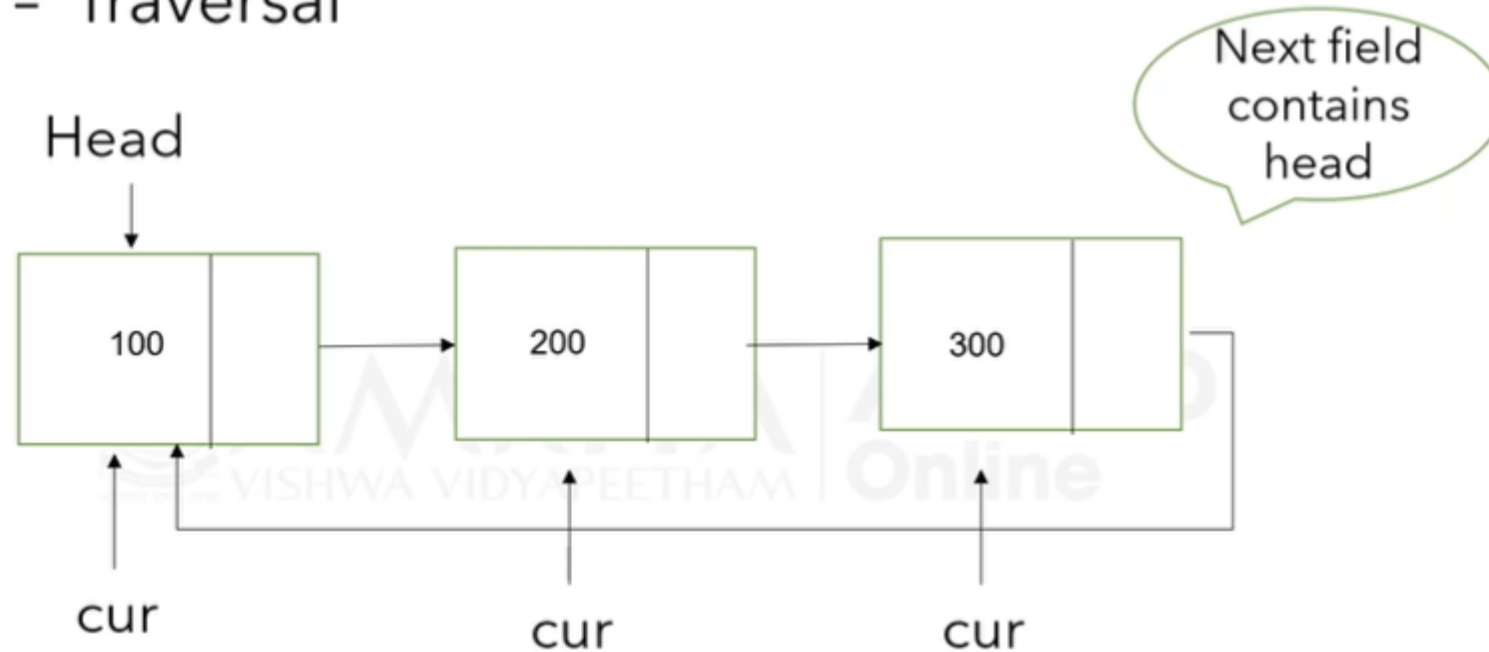
Circular Linked List

Operations – Traversal



Circular Linked List

Operations – Traversal



Circular Linked List

```
public void traverse() {  
    CNode cur=head;  
    if(cur.getNext()==head) //Circular LL with single node  
    {System.out.println(cur.getData());}  
    else {  
        while(cur.getNext()!=head) {  
            System.out.println(cur.getData());  
            cur=cur.getNext();  
        }  
        System.out.println(cur.getData()); //print last node  
        :  
    }  
}
```

Summary

Circular Linked List

Insertion, Traversal



Reference

Fundamentals of Data Structures by Ellis Horowitz and Sartaj Sahni



Objectives

Doubly Linked List

Doubly Linked List Operations

**Advantages and disadvantages of
Doubly Linked List**

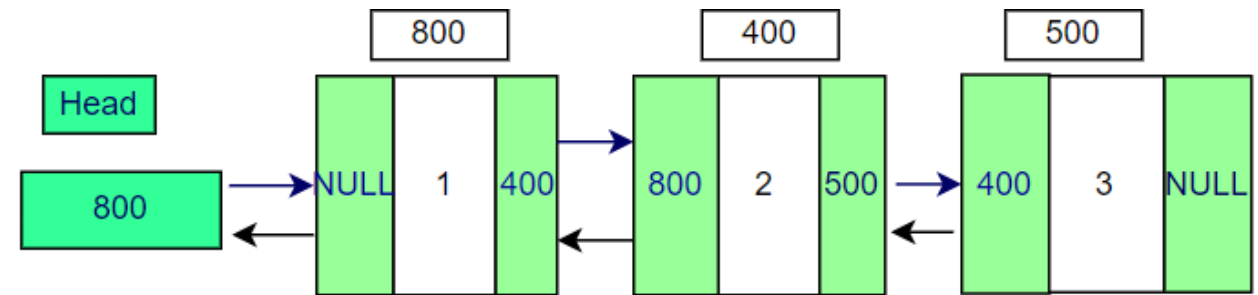
Linked List

- Linear Data Structure similar to Linked List.
- Stores collection of items.
- Follows a linked representation.(successive items are not stored sequentially in adjacent locations as in arrays, items may be placed any where in the memory.)
- Each element of the list stores the address of next element and previous element, for traversing list from the beginning and end.

Doubly Linked List

Head - Stores the address of the first node.

Node - Stores data, address to next node, address to previous node.



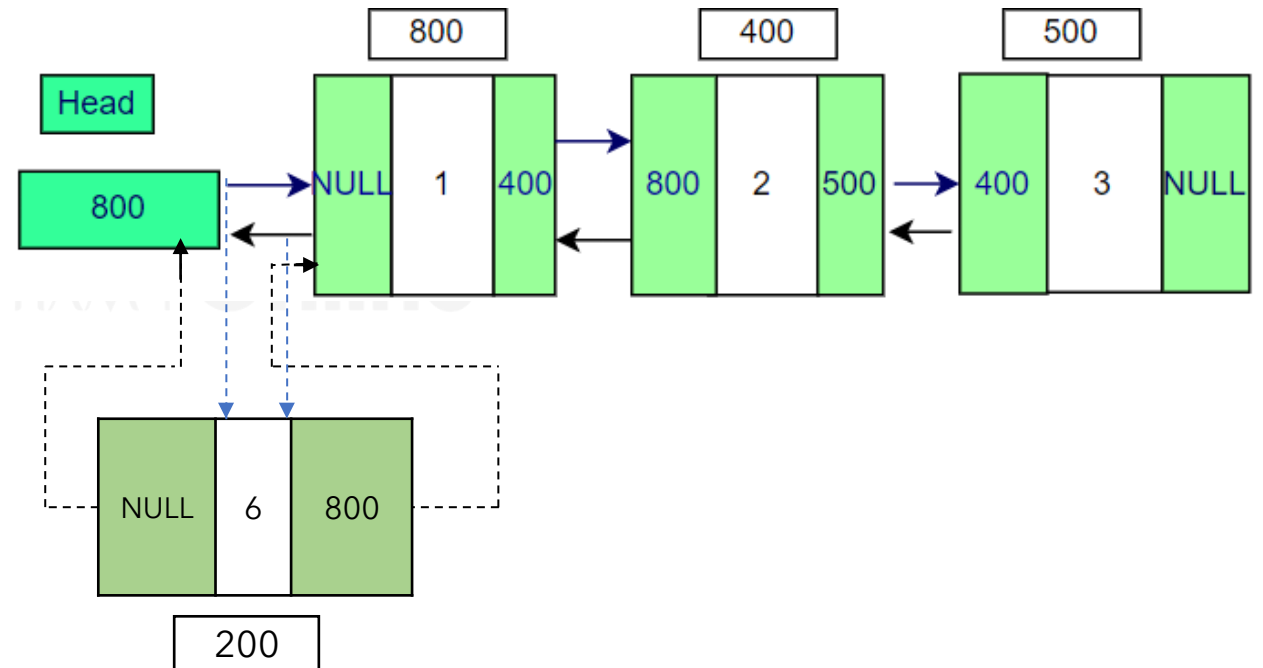
Doubly Linked List: Operations

- **Insertion:**
 - Insert at the beginning of the list.
 - Insert at the end of the list.
 - Insert item into a particular position of list.
- **Deletion:**
 - Deleting the first node.
 - Deleting the last node.
 - Deleting an intermediate node.
- **Traverse()**
 - Forward traversal.
 - Backward traversal.

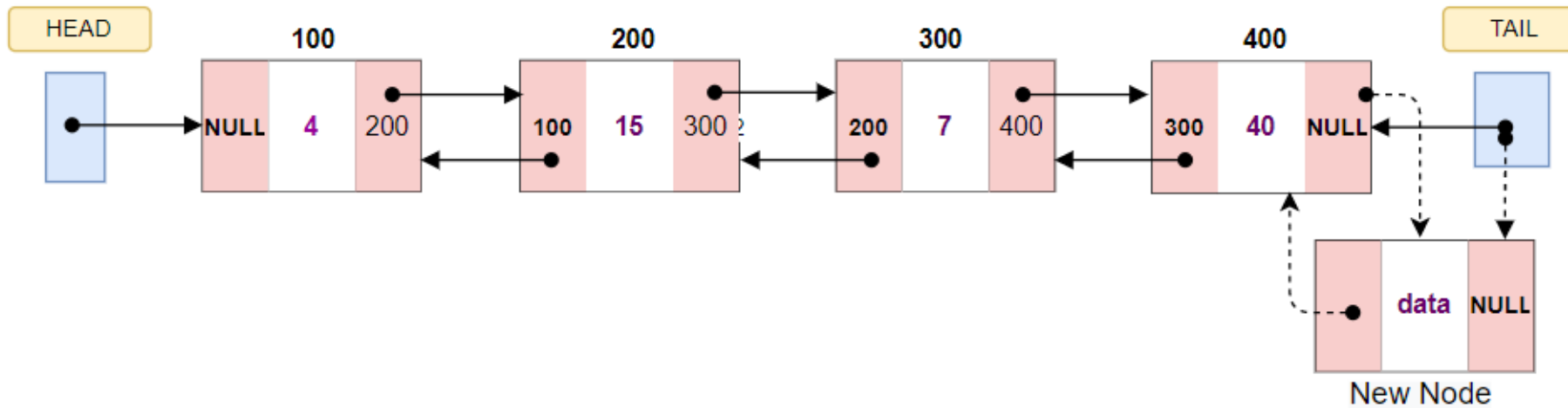
Inserting into a doubly linked list

Insert at the head

```
newNode->next=head  
newNode->prev=NULL  
head->prev=newnode  
head=newnode
```

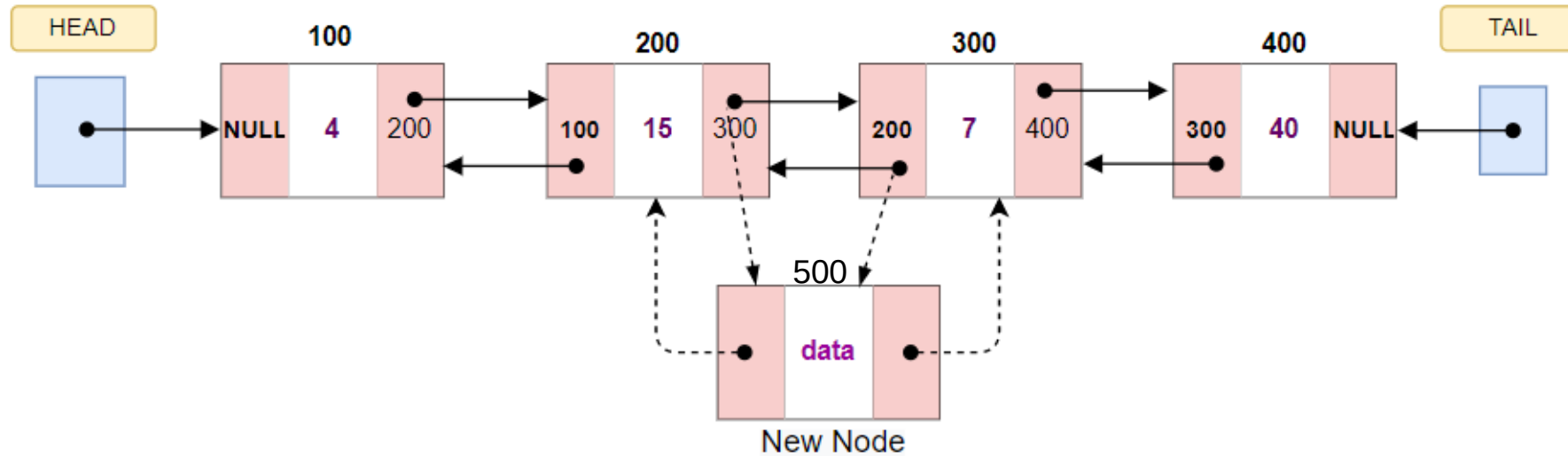


Inserting at the end



NewNode ->next= NULL
NewNode->previous=tail
tail->next=NewNode
tail->NewNode

Inserting into doubly Linked List

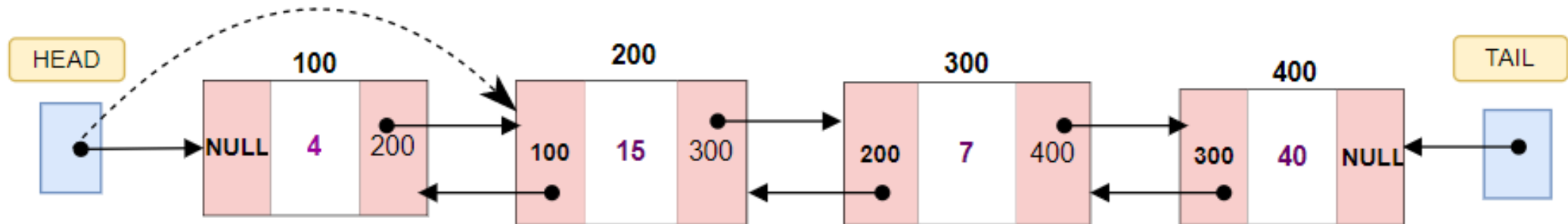


```
newNode->previous= temp;  
newNode->next = temp->next  
temp->next->previous=newNode;  
temp->next= newNode;
```

```
k=1  
While(k!=position)  
    temp=temp->next  
    k++
```

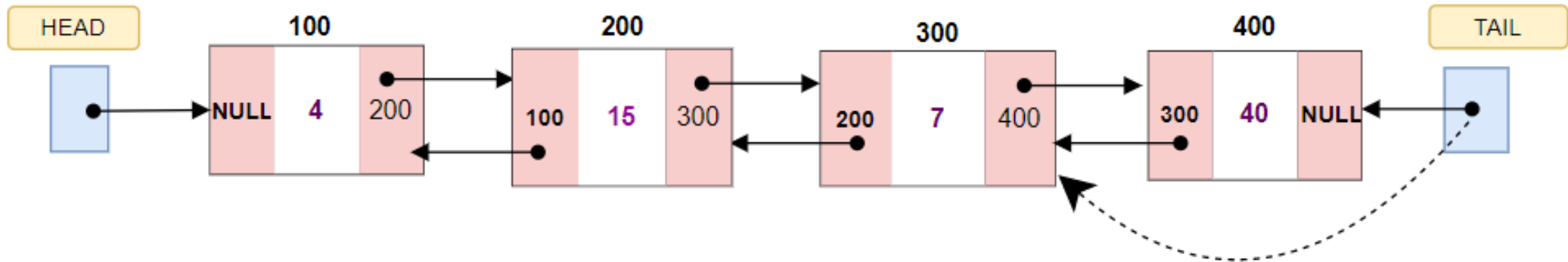
Deleting first node of DLL

```
head=head->next;  
head->prev=NULL;
```

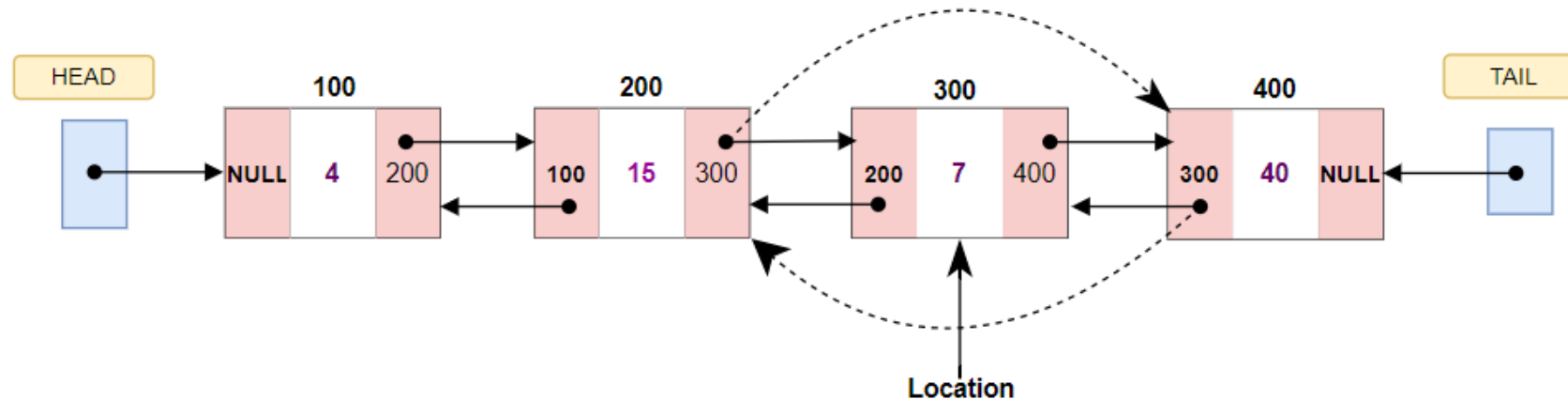


Deleting last node of DLL

tail->prev->next=NULL
tail=tail->prev



Deleting from Linked List



`temp ->next->prev = temp ->prev;`
`temp ->prev->next = temp ->next;`

`k=1`
`While(k!=position)`
`temp=temp->next`
`k++`

Traversal

Forward traversal

```
temp=head  
while(temp->next!=NULL)  
    temp=temp->next
```

Backword traversal

```
temp=tail  
while(temp->prev!=NULL)  
    temp=temp->prev
```

Doubly Linked List Advantages

- ❖ The doubly linked list can be traversed easily in forward as well as backward direction
- ❖ You can quickly add the new node just by changing the pointers.
- ❖ For delete operation since we have previous as well as next pointers, the deletion is easier.
- ❖ Need not traverse the whole list to find the previous node as in case of the singly linked list.

Doubly Linked List Disadvantages

- ❖ The previous pointer, additional memory space is required to store this pointer along with the next pointer and data item.
- ❖ Imposing operational overhead.

Summary

Doubly Linked List Operations

Advantages and disadvantages
of Doubly Linked List



References

- Michael T Goodrich and Roberto Tamassia and Michael H Goldwasser, "Data Structures and Algorithms in Java", John Wiley publication, 2013.
- Clifford A. Shaffer, "Data Structures and Algorithm Analysis", Third Edition, Dover Publications, 2012.



AHEAD Online

Data Structures and Algorithms

Geetha Lekshmy V

Harsha Raj

Department of Computer Science
Amrita Vishwa Vidyapeetham

Objectives

Tree

Tree - Applications

Binary Tree

Types of Binary Tree

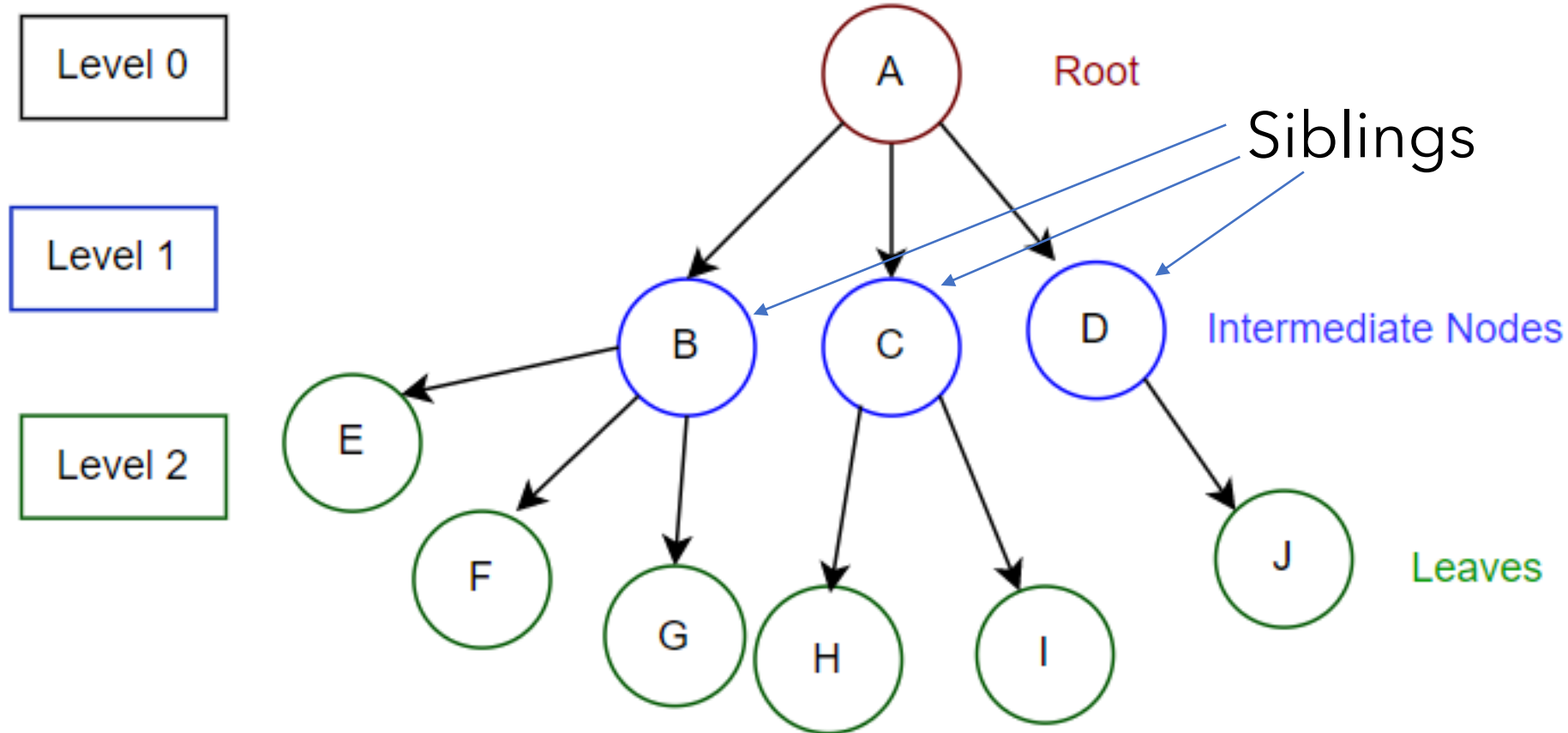
Binary Tree Properties



Tree

- Non-Linear Data Structure.
- Set of nodes – One node is distinguished as root and other nodes are partitioned into disjoint set called subtrees of the root.
- Tree is similar to linked list but each node refers to a number of nodes.

Tree



Height of the Tree -2

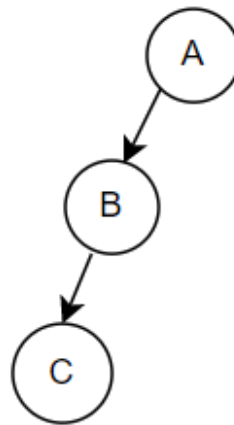
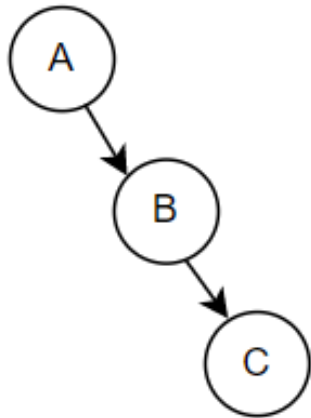
Root has degree-3

Tree

- Set of all nodes at a given depth is called level of a tree.
Root is at level 0.
- The depth of a node is the length of the path from root to the node
- Height of a tree – length of the path from the root to deepest node.
- A tree with only one node has a height of zero.

Tree

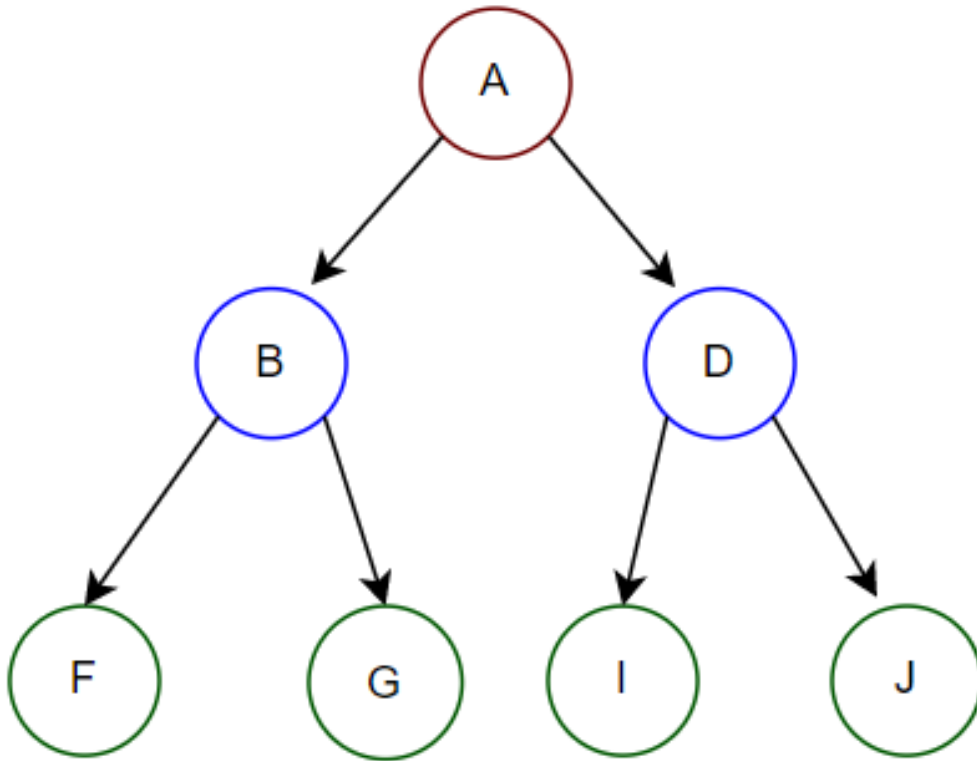
- Skewed Tree - Every node has only one child, except the leaf node.
- Right Skewed Tree
- Left Skewed Tree



Tree Data Structure - Applications

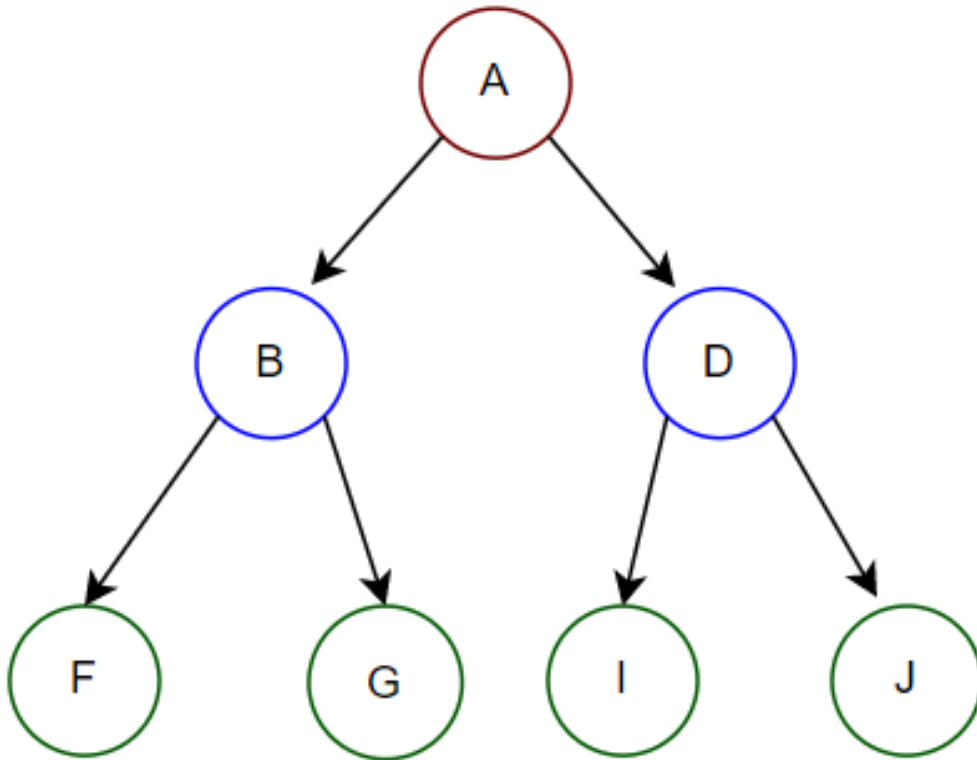
- File system in Operating System
- B-Tree, B+-Tree - indexing in Data bases
- Syntax Tree - in compilers
- Document Object Model(DOM)

Binary Tree



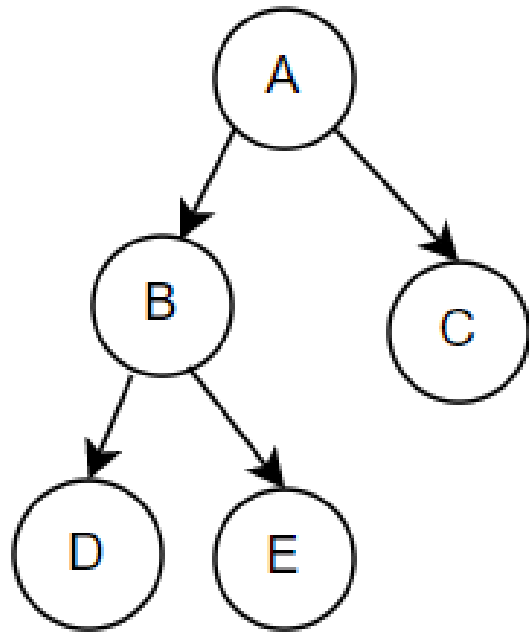
- Finite set of nodes consisting of root, and two subtrees called left sub tree and right subtree.
- Each node of a binary tree has zero, one or two children.

Binary Tree –Full Binary Tree



- Each node in the tree has exactly two children and all the leaves at the same level.

Binary Tree –Complete Binary Tree



- Complete Binary Tree – All the levels are completely filled except possibly the lowest one which is filled from the left.

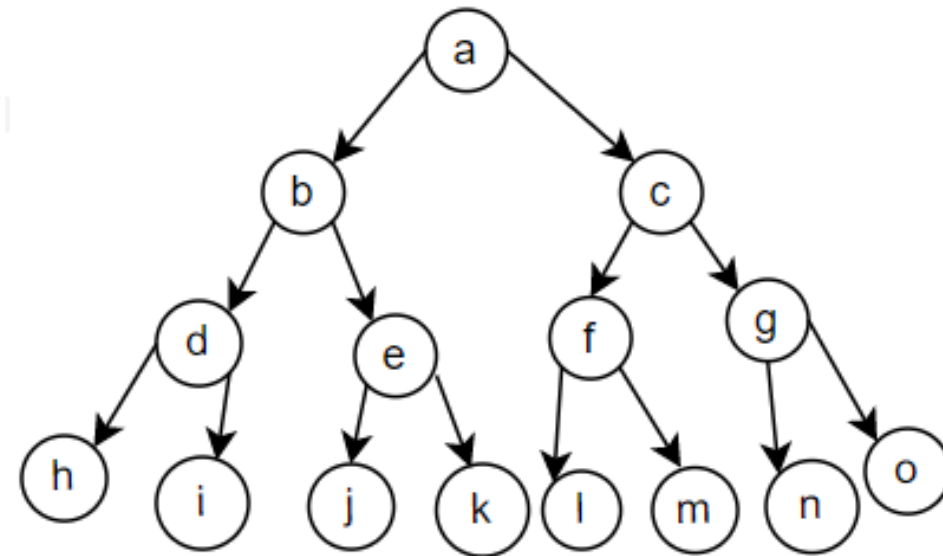
Binary Tree – Properties

- The number of nodes at level h in a **full** binary tree = 2^h
- The number of nodes in a **full** binary tree is $2^{h+1} - 1$.

Since, there are h levels we need to add all nodes at each level,

$$2^0 + 2^1 + 2^2 + \dots + 2^h = 2^{h+1} - 1.$$

- The number of nodes n in a Complete binary tree is between 2^h (minimum) and $2^{h+1} - 1$ (maximum).



Level-0 , 2^0 nodes

Level-1, 2^1 nodes

Level-2, 2^2 nodes

Level-3, 2^3 nodes

Summary

Multiway Trees

Binary Trees and its Properties



Reference

Fundamentals of Data Structures by Ellis Horowitz and Sartaj Sahni

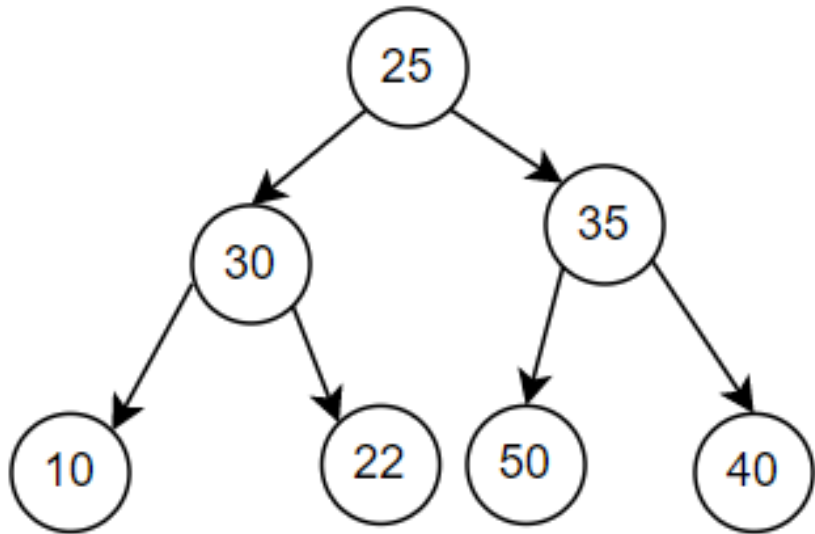


Objectives

Sequential Representation of Binary Trees



Implementing Binary Tree Using Arrays



25	30	35	10	22	50	40
0	1	2	3	4	5	6

For a node at index i ,

Index of Left Child

$$\text{Left}(i) \rightarrow 2*i + 1$$

Index of RightChild

$$\text{Right}(i) \rightarrow 2*i + 2$$

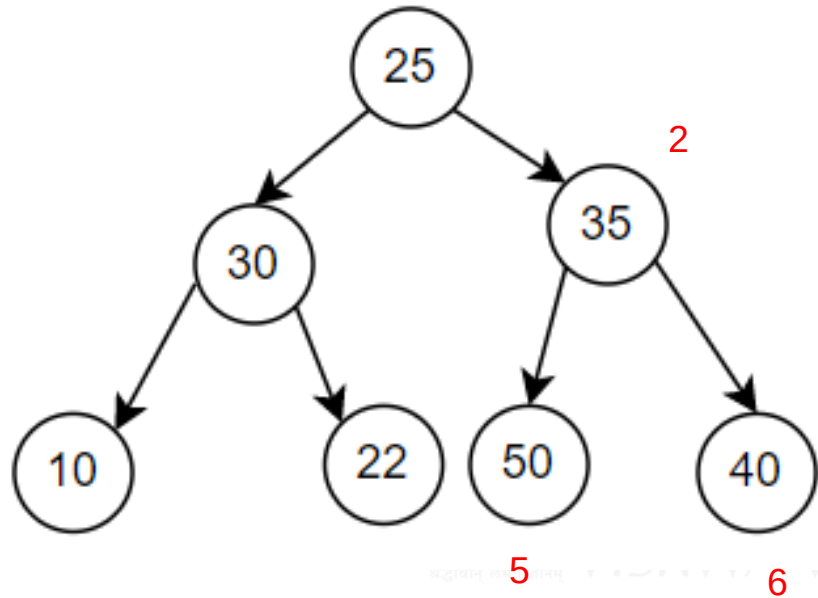
$i=0$ (root node)

$$\text{Left}(0) \rightarrow 2*0 + 1 = 1$$

$$\text{Right}(0) \rightarrow 2*0 + 2 = 2$$



Implementing Binary Tree Using Arrays



In which index will the Left Child of 35 be stored?

35 is stored in index 2

$$\text{Left}(2) \rightarrow 2*2 + 1 = 5$$

$$\text{Right}(2) \rightarrow 2*2 + 2 = 6$$

25	30	35	10	22	50	40
0	1	2	3	4	5	6



Objectives

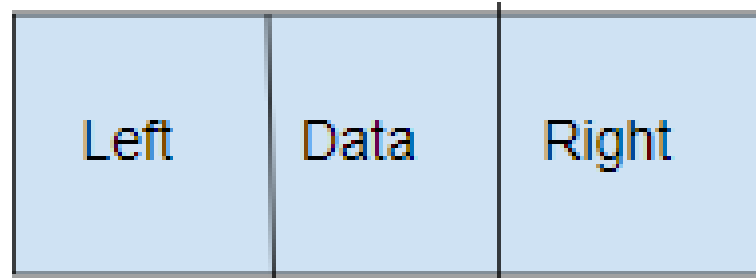
Linked Representation of
Trees

Binary Tree Operations –
Traversal, Insertion

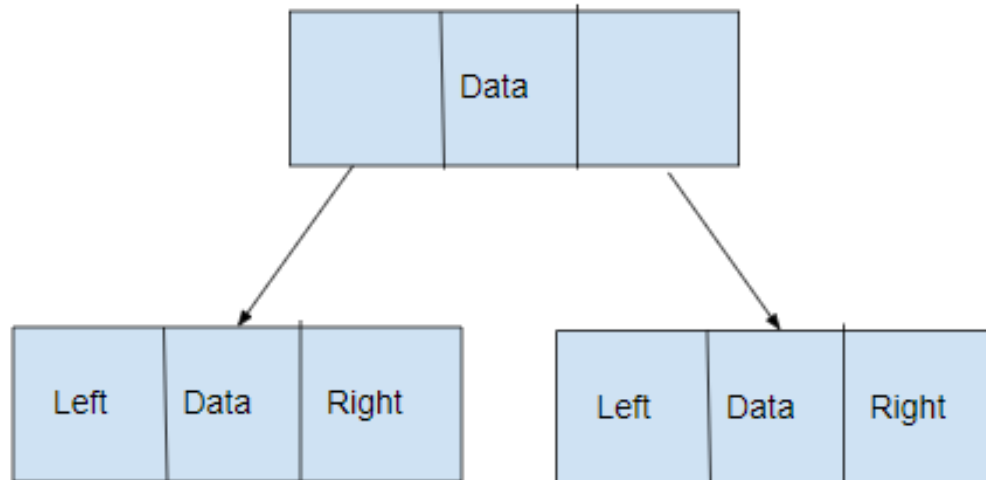
AMRITA | AHEAD
VISHWA VIDYAPEETHAM | Online

Node of a binary Tree

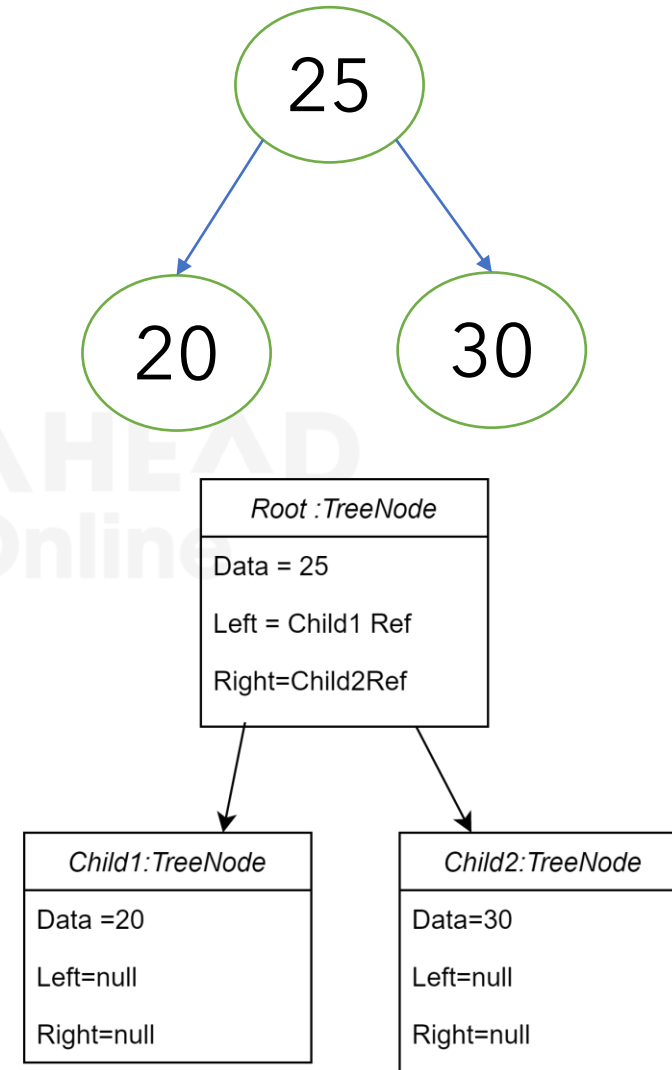
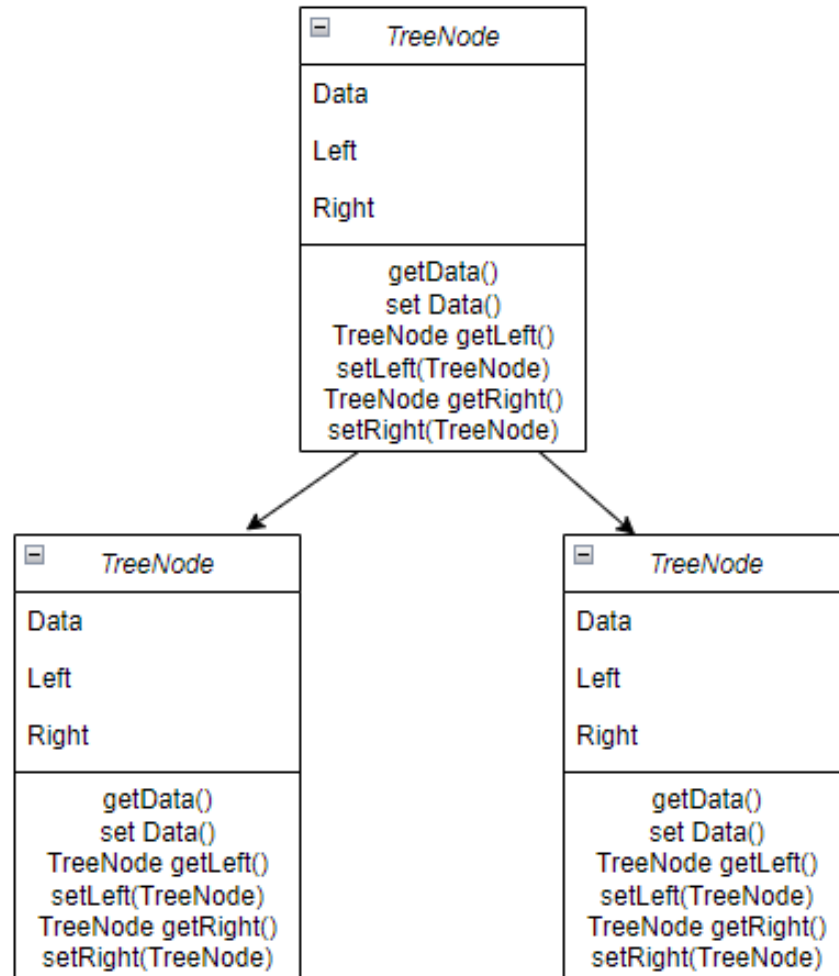
Structure of a node



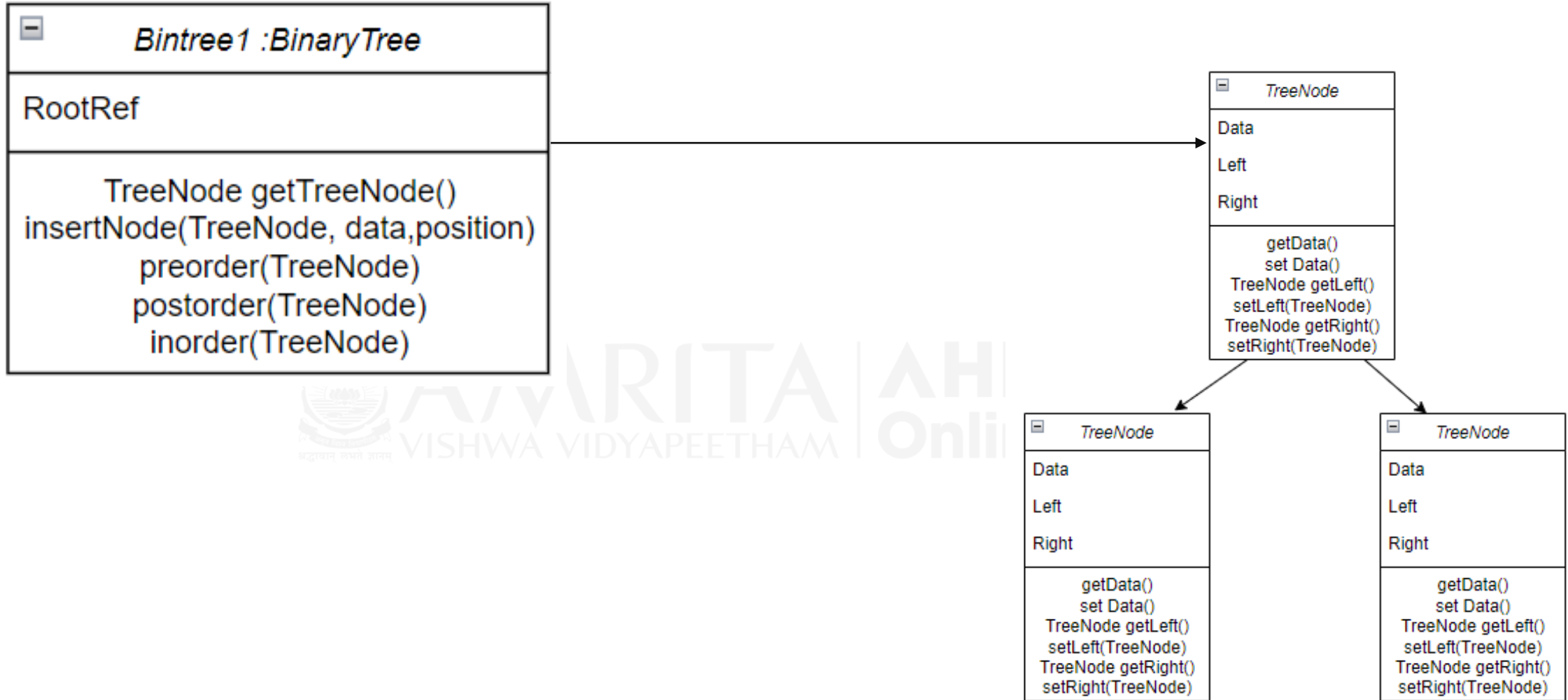
Node with two child nodes



Structure of a binary Tree



Structure of a binary Tree



Binary Tree Node in Java

```
class TreeNode{
    private int data;                //Store data values
    private TreeNode left;           //reference to left child
    private TreeNode right;          //reference to right child
    public TreeNode() {data=0;left=null;right=null;}
    public TreeNode(int d) {data=d;left=null;right=null;}
    public TreeNode(int d, TreeNode l,TreeNode r)
    {data=d;left=null;right=null;}
    public int getData() {return data;}
    public void setData(int d) {data=d;}
    public void setLeft(TreeNode n) {left=n;}
    public void setRight(TreeNode n) {right=n;}
    public TreeNode getLeft() {return left;}
    public TreeNode getRight() {return right;} }
```

Binary Tree

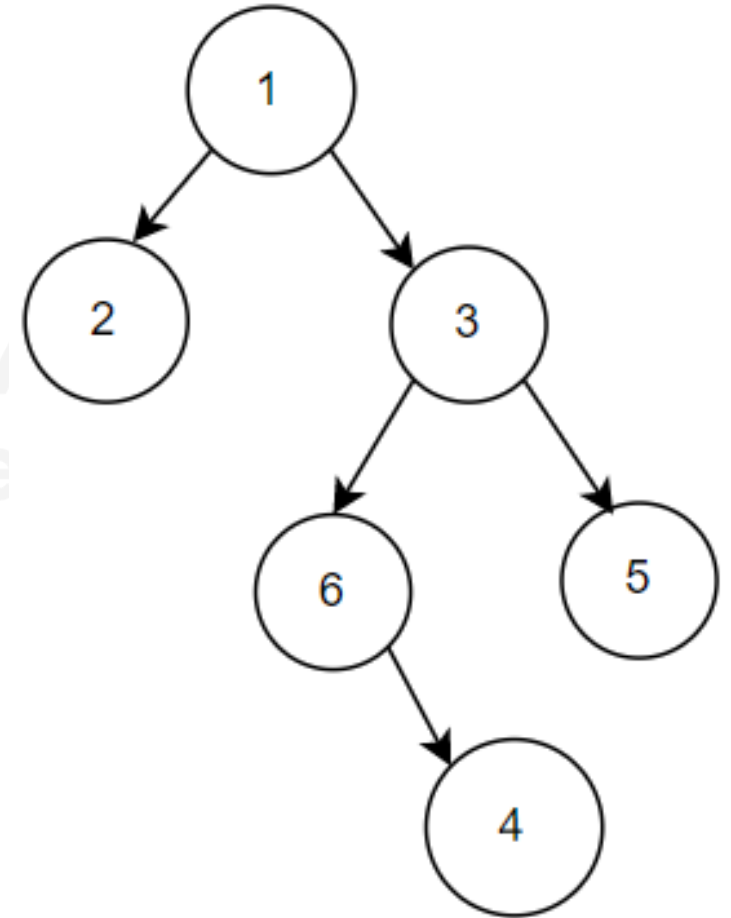
```
class BinTree
{
    private TreeNode root;
    public BinTree() {root=null;}
    public TreeNode getTreeNode() {return root;}
    public void insertNode(TreeNode parent, int data,char pos) {..}
    void inorder(TreeNode r){..}
    void postorder(TreeNode r){..}
    void preorder(TreeNode r){..}
}
```

Binary Tree Traversal – Pre-order

Visit the root.

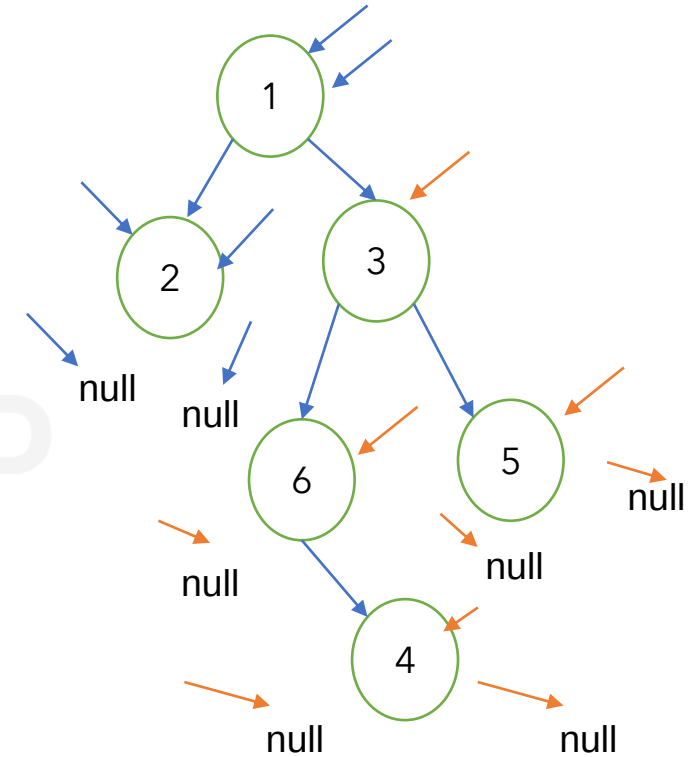
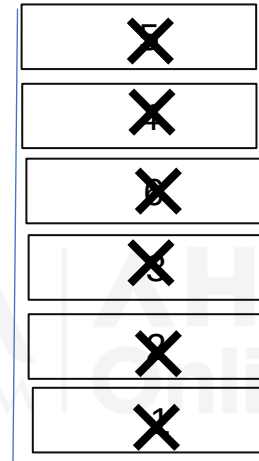
Visit left subtree in pre-order.

Visit right subtree in pre-order.



Binary Tree Traversal – Pre-order- Example

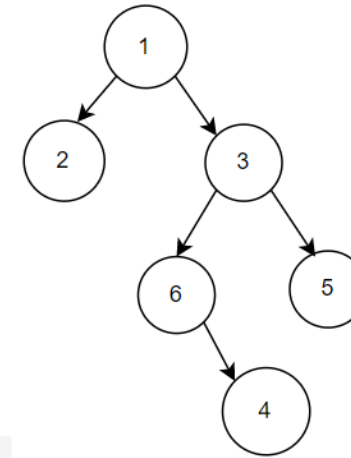
```
S=Stack()
r=root
while (1){
while (r!=null){
    Visit r
    Print r
    Push(S,r)
    r=r.getLeft();
}
If (S.empty() )break;
r=pop();
r=r.getRight();
}
```



1 2 3 6 4 5

Binary Tree Traversal – Pre-order- Recursive

```
preorder(TreeNode r){  
    If (r!=null){  
        print(r.getData())  
        preorder(r.getLeft())  
        preorder(r.getRight())  
    }  
}
```



AMRITA
VISHWA VIDYAPEETHAM

AHEAD
Online



Binary Tree Insertion

```
void insertNode(TreeNode parent,int data,char pos{
    TreeNode newnode = new TreeNode(data);
    If(parent == null) {
        root = newnode;
    }else {
        if(pos=='L') { parent.setLeft(newnode);}
        else { parent.setRight(newnode); }
    }
}
```

Summary

Implement trees using Linked List

Pre-order traversal (Recursive & Non -Recursive), Binary Tree Insertion



Reference

Data Structures and Algorithms – Narasimha Karumanchi

